Experiment Title: Implementation of Shortest Job First (SJF) Non-Preemptive CPU Scheduling Algorithm

Topic: Non-Preemptive CPU Scheduling with Arrival Time in C

1. Problem

Implement the Shortest Job First (SJF) non-preemptive CPU scheduling algorithm in C. Given the arrival times (AT) and burst times (BT) for a set of processes, schedule the arrived process with the shortest burst time, break ties using FCFS order, handle CPU idle time, and compute Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), and average metrics.

2. Theoretical Solution

SJF non-preemptive scheduling selects the ready process ($AT_i \leq currentTime$) with the minimum burst time. Once scheduled, it runs to completion without preemption. Equal burst times are broken by arrival order. If no process is ready, the CPU remains idle until the next arrival.

2.1 Formulas

For each process i:

- $CT_i = ST_i + BT_i$
- $TAT_i = CT_i - AT_i$
- $WT_i = TAT_i - BT_i$
- $Avg_TAT = \frac{1}{n}\sum TAT_i, \quad Avg_WT = \frac{1}{n}\sum WT_i$

2.2 Algorithmic Steps

1. Read process count n, arrival times AT, and burst times BT. Sort initially by arrival time.
2. From uncompleted processes with $AT_i \leq currentTime$, select the one with minimum BT (tie-break with smaller AT).
3. Advance $currentTime$ by its burst time, calculate CT, TAT, WT, and mark it completed.
4. If no process is arrived, increment $currentTime$ by 1.
5. Repeat until all processes finish, then display results and averages.

3. Expected Input/Output (I/O) Set

Expected input and scheduling results for $n = 4$ processes:

Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turnaround Time (TAT)	Waiting Time (WT)
P1	0	7	7	7	0
P2	2	4	12	10	6
P3	4	1	8	4	3
P4	5	4	16	11	7

Expected Averages: Average Turnaround Time = 8.00, Average Waiting Time = 4.00.

4. Source Code

Complete C implementation of the Non-Preemptive SJF scheduling algorithm:

```c
#include <stdio.h>

int main() {
    int n;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    int process[n], arrival[n], burst[n];
    int completion[n], turnaround[n], waiting[n];
    int is_completed[n];

    int i, j, temp, pos;
    int current_time = 0;
    int completed = 0;

    float totalTAT = 0;
    float totalWT = 0;
    float averageTAT, averageWT;

    // Input process details
    for (i = 0; i < n; i++) {
        process[i] = i + 1;

        printf("Enter arrival time of process %d: ", process[i]);
        scanf("%d", &arrival[i]);

```

```
        printf("Enter burst time of process %d: ", process[i]);
        scanf("%d", &burst[i]);

        completion[i] = -1;
        is_completed[i] = 0;
    }

    // Sorting processes based on arrival time
    for (i = 0; i < n; i++) {
        pos = i;

        for (j = i + 1; j < n; j++) {
            if (arrival[j] < arrival[pos]) {
                pos = j;
            }
        }

        // Swap arrival time
        temp = arrival[i];
        arrival[i] = arrival[pos];
        arrival[pos] = temp;

        // Swap burst time
        temp = burst[i];
        burst[i] = burst[pos];
        burst[pos] = temp;

        // Swap process ID
        temp = process[i];
        process[i] = process[pos];
        process[pos] = temp;
    }

    // Apply Shortest Job First (Non-Preemptive)
    while (completed != n) {

        int min_burst = 9999;
        int index = -1;

        // Find the shortest available process
        for (i = 0; i < n; i++) {

            if (arrival[i] <= current_time && is_completed[i] == 0) {

                if (burst[i] < min_burst) {
                    min_burst = burst[i];
                    index = i;
                }

                // If burst times are equal, select the one
                // that arrived first
                else if (burst[i] == min_burst &&
                         arrival[i] < arrival[index]) {
                    index = i;
                }
            }
        }

        // If a process is available
        if (index != -1) {

            completion[index] = current_time + burst[index];
            turnaround[index] = completion[index] - arrival[index];
            waiting[index] = turnaround[index] - burst[index];

            // Add to totals
            totalTAT = totalTAT + turnaround[index];
            totalWT = totalWT + waiting[index];

            // Update current time
            current_time = completion[index];

            is_completed[index] = 1;
            completed++;
        }

        // If no process has arrived yet
        else {
            current_time++;
        }
    }

    // Calculate averages
    averageTAT = totalTAT / n;
    averageWT = totalWT / n;
```

```c

    // Printing the final process table
    printf("\nProcess\tArrival Time\tBurst Time\tCompletion Time\tTurnaround Time\tWaiting Time\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t%d\t\t%d\t\t%d\n",
               process[i],
               arrival[i],
               burst[i],
               completion[i],
               turnaround[i],
               waiting[i]);
    }

    // Print total and average values
    printf("\nTotal Turnaround Time = %.0f", totalTAT);
    printf("\nTotal Waiting Time    = %.0f", totalWT);

    printf("\nAverage Turnaround Time = %.2f", averageTAT);
    printf("\nAverage Waiting Time    = %.2f\n", averageWT);

    return 0;
}
```

5. Execution of Code

The program was compiled with **gcc** and executed in Linux terminal:

ratul@firefly: ~/OS-Lab — bash

```
ratul@firefly:~/OS-Lab$ gcc sjf.c -o sjf
ratul@firefly:~/OS-Lab$ ./sjf
Enter the number of processes: 4
Enter arrival time of process 1: 0
Enter burst time of process 1: 7
Enter arrival time of process 2: 2
Enter burst time of process 2: 4
Enter arrival time of process 3: 4
Enter burst time of process 3: 1
Enter arrival time of process 4: 5
Enter burst time of process 4: 4
Process  Arrival Time  Burst Time  Completion Time  Turnaround Time  Waiting Time
P1       0             7           7                7                0
P2       2             4           12               10               6
P3       4             1           8                4                3
P4       5             4           16               11               7
Total Turnaround Time = 32
Total Waiting Time    = 16
Average Turnaround Time = 8.00
Average Waiting Time    = 4.00
ratul@firefly:~/OS-Lab$ _
```

Verification: The terminal execution results match the calculated completion times, turnaround times, waiting times, and averages, confirming the correctness of the SJF non-preemptive algorithm.